

DOCUMENTACIÓN TÉCNICA

Hacienda El Encanto — Portal de Eventos

CLIENTE

Hacienda El Encanto — Juan Carlos Pulido

DESARROLLADOR

Ing. Jeisson Rincón

FECHA

29 de julio de 2026

Última actualización: 2026-07-29

Estado: Producción activa en <https://www.hacienda-encanto.com>

1. Resumen del proyecto

Campo	Valor
Nombre	Hacienda El Encanto — Portal de eventos
Descripción	Sitio web público + portal privado para la gestión integral de eventos sociales y empresariales en la Hacienda El Encanto (Cota, Cundinamarca). Incluye catálogo público, formulario de contacto con asignación automática de asesores, y un portal multi-rol para clientes, planners, asesores, staff y administración.
Cliente	Hacienda El Encanto
Dominio de producción	https://www.hacienda-encanto.com
Repositorio	<code>develop</code> (rama de trabajo) <code>main</code> (producción)
Fecha de lanzamiento	Julio 2026
ID de proyecto Supabase	<code>oewqyckeqlrpjbjevap</code>

2. Stack tecnológico

Capa	Tecnología	Versión
Framework	Next.js	16.2.9
Lenguaje	TypeScript	v5
UI	React	19.2.4
Estilos	Tailwind CSS	v4 (sintaxis <code>@theme inline {}</code>)
Base de datos	Supabase (PostgreSQL)	<code>@supabase/supabase-js</code> v2.108.2
Auth + SSR	<code>@supabase/ssr</code>	v0.12.0
Deploy	Vercel	—
PDF	<code>@react-pdf/renderer</code>	v4.5.1
Drag & drop	<code>@dnd-kit/core</code> + <code>sortable</code>	v6 / v10
Formularios	<code>react-hook-form</code> + <code>zod</code>	v7 / v4.4.3
Animaciones	<code>framer-motion</code>	v12

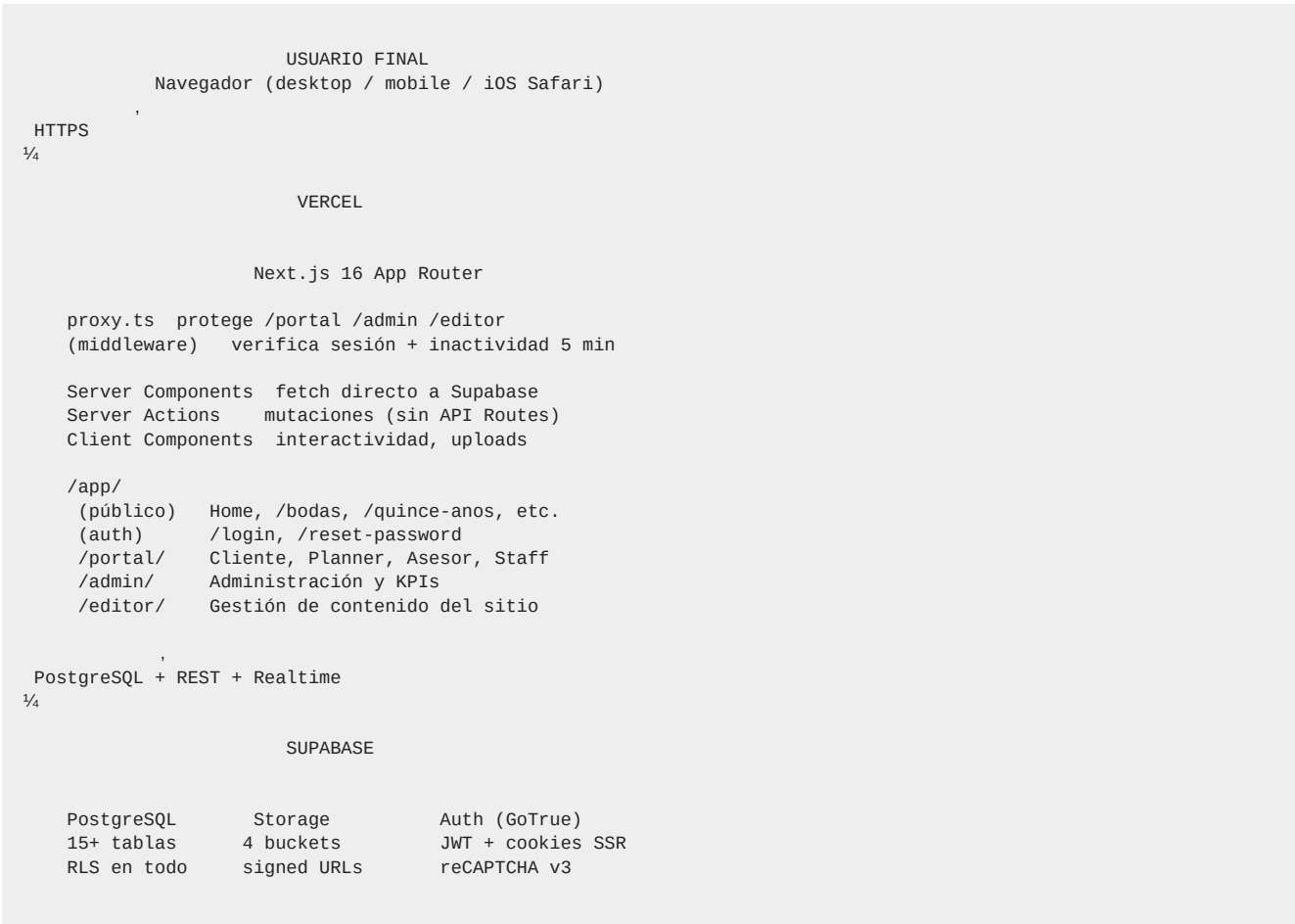
Iconos	lucide-react	v1.21.0
Fechas	date-fns	v4.4.0
Package manager	pnpm	(exclusivo — no usar npm)

Fuentes: Cormorant Garamond (self-hosted vía `next/font`)

Paleta de marca:

Variable CSS	Uso
--rojo	Color primario
--dorado	Acentos y detalles
--crema	Fondos claros
--negro	Texto y overlays
--verde-bosque	Elementos de naturaleza
--blush	Acentos femeninos

3. Arquitectura del sistema



```
pg_cron: sync_completed_bookings() — diario
```

¼

SERVICIOS EXTERNOS

CallMeBot (WhatsApp API) — notificaciones de nuevos contactos
Google reCAPTCHA v3 — protección de formularios

Patrón de upload (signed URL — crítico)

Los archivos **nunca pasan por Vercel** (límite de 4.5 MB en el body). El flujo es:

```
Cliente 1. SA "request" $Supabase Storage (genera signed upload URL)
Assigned URL

Cliente 2. PUT directo $Supabase Storage (archivo va directo)

Cliente 3. SA "confirm" PostgreSQL (inserta/actualiza registro en BD)
```

4. Base de datos

Proyecto Supabase: `oewqyckeqolrpbjevap`. Todas las tablas tienen RLS habilitado.

4.1 Tablas principales

| profiles

Extiende `auth.users` (relación 1:1). Se crea automáticamente al registrar un usuario via trigger `on_auth_user_created`.

Campo	Tipo	Descripción
id	uuid PK	Mismo UUID que <code>auth.users.id</code>
email	text	Email del usuario
full_name	text	Nombre completo
phone	text	Teléfono (privado — solo admin)
cc	text	Cédula / NIT
role	user_role enum	Rol del usuario
avatar_url	text	URL del avatar en Storage
is_active	boolean	Si el usuario está activo
last_active_at	timestampz	Última actividad (para inactividad 5 min)
callmebot_api_key	text	API key personal de CallMeBot (as-esores)
created_at / updated_at	timestampz	Auditoría

bookings

Reservas de eventos. Una reserva = un evento.

Campo	Tipo	Descripción
id	uuid PK	—
client_id	uuid FK ' profiles	Cliente dueño del evento
space_id	uuid FK ' spaces	Salón o zona del evento
event_date	date	Fecha del evento
event_start_time / event_end_time	time	Horario (validación solapamiento en SA)
guest_count	int	Número de invitados (opciones 30–150 de 10 en 10)
event_type	text	Tipo: boda, quinceañera, empresarial, revelacion
status	booking_status	`pending \ confirmed \ cancelled \ completed`
notes	text	Observaciones internas
valor_total / valor_anticipo	numeric	Valores financieros del contrato
fecha_segundo_abono / fecha_tercer_abono	date	Fechas de abonos
capilla	boolean	Si incluye uso de capilla
contract_items	jsonb	Ítems del contrato (dj, decoración, etc.)

contract_locked	boolean	true cuando el cliente aprueba el contrato
total_amount	numeric	Monto total de la reserva

spaces

Salones y zonas de la hacienda, gestionados por el admin.

Campo	Tipo	Descripción
id	uuid PK	—
name	text	Nombre del espacio
slug	text unique	Slug para URL
capacity_min / capacity_max	int	Rango de capacidad
area_m2	numeric	Área en metros cuadrados
amenities	jsonb	Lista de amenidades
is_active	boolean	Visibilidad pública

packages

Paquetes de servicio. Sin campo price público (eliminado en migración 20260722).

Campo	Tipo	Descripción
id	uuid PK	—
name	text	Nombre del paquete
description	text	Descripción
includes	jsonb	Lista de ítems incluidos
is_active / sort_order	—	Control de visibilidad y orden

payments

Pagos manuales, sin pasarela de pagos.

Campo	Tipo	Descripción
booking_id	uuid FK	Reserva asociada
amount	numeric	Monto pagado
payment_date	date	Fecha del pago
payment_method	enum	`transferencia` \ `efectivo` \ `cheque` \ `otro`
reference_number	text	Número de referencia bancaria
receipt_url	text	URL del comprobante en Storage
confirmed		

boolean	Si el admin confirmó el pago	
concept	text	Concepto del pago
recorded_by	uuid FK	Staff que registró el pago

documents

Documentos por reserva (contratos, comprobantes).

Campo	Tipo	Descripción
booking_id	uuid FK	Reserva asociada
type	document_type	`contrato` \ contrato_firmado`
title	text	Nombre del archivo
file_url	text	URL en Storage
created_by	uuid FK	Quién subió el documento

contact_messages

Formulario de contacto público. Acceso anónimo para INSERT.

Campo	Tipo	Descripción
name / email	text	Datos del prospecto
phone / whatsapp	text	Teléfonos (whatsapp requerido)
event_type / event_date	text / date	Tipo y fecha de evento deseado
guest_count	int	Invitados estimados
message	text	Mensaje libre
status	contact_status	`unread` \ read` \ replied` \ en_proceso`
assigned_asesor_id	uuid FK	Asesor asignado por round-robin

service_order_sections / service_order_items

Orden de servicio estructurada en secciones e ítems. Modelo dos actores: filled_by = 'planner' o 'client'.

Campo clave	Descripción
section (en sections)	Nombre de la sección (ej. "Cabecera", "Música")
sort_order	Orden de visualización
event_type	Tipo de evento o 'all'
label / value / field_type / filled_by	Definición de cada ítem

- playlists

Música por sección del evento. Una fila por (`booking_id`, `section`).

Campo	Descripción
<code>section</code>	Sección de playlist (enum: <code>entrada_novio</code> , <code>vals_pareja</code> , etc.)
<code>song_name</code> / <code>song_url</code>	Canción y URL (Spotify, YouTube, etc.)
<code>no_aplica</code>	Centinela: si la sección no aplica para este evento

| guest_tables

Distribución de invitados. Cada fila = versión histórica del archivo Excel subido por el cliente.

| calendar_events

Calendario central. `booking_id` nullable (permite bloquear fechas antes de reserva formal).

| client_activities

Timeline de actividades para el cliente (hitos, tareas, recordatorios). CRUD inline por el planner.

| salon_maps

Mapas de salón por capacidad. El planner sube imágenes; el sistema busca el mapa adecuado según `guest_count`.

| testimonials

Testimonios de clientes. `is_published` controla visibilidad pública. Incluye `photo_url`.

| gallery_images

Imágenes de galería. Secciones Publicadas (máx 8/categoría, reordenables con DnD) y Archivadas.

| hero_videos

Videos del hero. `event_type = NULL` 'Home'. `event_type = 'boda'` 'página /bodas, etc.

| site_content

CMS ligero. Pares `key { title, content, data }`. Claves principales:

Prefijo de clave	Uso
<code>img_card_*</code>	Imágenes de cards de eventos en Home
<code>img_nosotros</code> , <code>img_servicio_*</code>	Secciones del Home
<code>hacienda_*</code>	Datos de la hacienda para contratos
<code>contrato_clausula_1..12</code>	Cláusulas editables del contrato
<code>firma_representante</code>	URL de imagen de firma para PDF

| asesor_assignments

Round-robin de asignación de asesores. Una fila por asesor, con `total_assignments` y `last_assigned_at`.

login_attempts

Rate limiting de intentos de login fallidos. RLS `USING (false)` — solo `service_role` tiene acceso. Ventana de 15 min, máx 5 intentos, delay progresivo `min(2^(n-1), 16) * 1000ms`.

4.2 ENUMs

Enum	Valores
<code>user_role</code>	<code>client</code> , <code>staff</code> , <code>admin</code> , <code>wedding_planner</code> , <code>asesor_comercial</code> , <code>asesor_logistica</code> , <code>editor</code> , <code>gerente</code>
<code>booking_status</code>	<code>pending</code> , <code>confirmed</code> , <code>cancelled</code> , <code>completed</code>
<code>payment_method_type</code>	<code>transferencia</code> , <code>efectivo</code> , <code>cheque</code> , <code>otro</code>
<code>contact_status</code>	<code>unread</code> , <code>read</code> , <code>replied</code> , <code>en_proceso</code>
<code>document_type</code>	<code>contrato</code> , <code>contrato_firmado</code>
<code>service_order_status</code>	<code>pending</code> , <code>confirmed</code> , <code>in_progress</code> , <code>completed</code> , <code>cancelled</code>
<code>playlist_section</code>	<code>entrada_novio</code> , <code>entrada_novia</code> , <code>salida_recien_casados</code> , <code>entrada_salon</code> , <code>vals_pareja</code> , <code>vals_padres_novia</code> , <code>vals_padres_novio</code> , <code>playlist_cena</code> , <code>playlist_rumba</code> , <code>entrada_zona_verde</code> , <code>acompanamiento_zona_verde</code>

4.3 Funciones y triggers de PostgreSQL

Función	Tipo	Descripción
<code>set_updated_at()</code>	trigger	Actualiza <code>updated_at</code> en cada UPDATE
<code>handle_new_user()</code>	trigger	Crea fila en <code>profiles</code> al registrar usuario en <code>auth.users</code>
<code>is_admin()</code>	helper RLS	<code>true</code> si el usuario autenticado tiene rol <code>admin</code>
<code>is_staff_or_admin()</code>	helper RLS	<code>true</code> para <code>admin</code> , <code>staff</code> , <code>wedding_planner</code> , <code>asesor_comercial</code> , <code>asesor_logistica</code>
<code>is_planner_or_admin()</code>	helper RLS	<code>true</code> para <code>admin</code> , <code>wedding_planner</code>
<code>is_any_staff()</code>	helper RLS	<code>true</code> para cualquier rol no-client
<code>is_editor()</code>	helper RLS	<code>true</code> para <code>admin</code> , <code>editor</code>
<code>is_admin_or_gerente()</code>	helper RLS	<code>true</code> para <code>admin</code> , <code>gerente</code>
<code>initialize_service_order(p_booking_id)</code>	PL/pgSQL security definer	Crea orden de servicio desde plantilla; idempotente (borra y recrea). Pre-llena 6 campos de Cabecera desde el booking.
<code>sync_completed_bookings()</code>	PL/pgSQL security definer	Marca bookings vencidos como <code>completed</code> . Invocado oportunistamente + <code>pg_cron</code> diario.

check_and_update_last_active(p_user_id, p_timeout_minutes)	RPC	Verifica inactividad y actualiza last_active_at; retorna false si superó el timeout.
--	-----	--

5. Autenticación y roles

5.1 Flujo de autenticación

- **Proveedor:** Supabase Auth (GoTrue) con JWT
- **Estrategia SSR:** @supabase/ssr — cookies en lugar de localStorage
- **Middleware:** proxy.ts (exportado como proxy, no middleware.ts — breaking change Next.js 16)
- **Rutas protegidas:** /portal/*, /admin/*, /editor/*
- **Timeout de inactividad:** 5 minutos (configurable en proxy.ts)
- **reCAPTCHA v3:** En formularios de contacto y login. Badge oculto con texto legal visible.
- **Sin registro público:** /registro redirige a /login. Solo admin o planner crean cuentas.

5.2 Roles y permisos

Rol	Acceso portal	Descripción
client	/portal/dashboard	Cliente del evento. Accede a su propio evento.
wedding_planner	/portal/planner	Coordina eventos. Crea clientes, gestiona contratos y órdenes.
asesor_comercial	/portal/asesor-comercial	Gestiona contactos/leads asignados. Ve eventos próximos (15 días).
asesor_logistica	/portal/asesor-logistica	Ve eventos próximos (15 días).
staff	/portal/staff	DJ / animadores. Ve eventos activos y playlist (próximos 15 días).
editor	/editor/galeria	Gestiona contenido del sitio (galería, videos, imágenes, testimonios, paquetes).
gerente	/portal/gerente	Ve todos los eventos sin restricción de fecha.
admin	/admin/dashboard	Acceso total al sistema.

5.3 Restricción de visibilidad por fecha

UPCOMING_EVENT_WINDOW_DAYS = 15 (en src/lib/event-window.ts).

- **Sin restricción:** admin, gerente
- **Solo próximos 15 días:** wedding_planner, asesor_comercial, asesor_logistica, staff

5.4 Recuperación de contraseña

1. `/reset-password` llama `resetPasswordForEmail(email, { redirectTo: "https://www.hacienda-encanto.com/update-password" })` 2. Supabase envía email con `?token_hash=XXXX&type=recovery` 3. `/update-password` (standalone, fuera del grupo `(auth)`) llama `verifyOtp({ token_hash, type: "recovery" })` browser-side 4. Si OTP válido muestra form de nueva contraseña 5. `SAupdatePassword` redirect por rol

5.5 Cambio de contraseña por admin

`cambiarPassword` SA en `admin/usuarios.ts` usa `createAdminClient().auth.admin.updateUserById(userId, { password })`. Componente `CambiarPasswordButton` disponible en `/admin/usuarios` y `/admin/clientes/[clientId]`.

6. Variables de entorno

Copiar `.env.example` a `.env.local` y completar. **Nunca** commitar `.env.local`.

Variable	Ámbito	Descripción
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Cliente + Servidor	URL del proyecto Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Cliente + Servidor	Clave anon/pública de Supabase
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Solo servidor	Clave <code>service_role</code> (bypasa RLS). Solo en Server Actions y proxy.ts.
<code>CALLMEBOT_PHONE</code>	Solo servidor	Número de teléfono de prueba para CallMeBot (formato 57XXXXXXXXXX)
<code>CALLMEBOT_API_KEY</code>	Solo servidor	API key de prueba de CallMeBot
<code>CALLMEBOT_PHONE_CENTRAL</code>	Solo servidor	Número central oficial para notificaciones de nuevos contactos
<code>CALLMEBOT_API_KEY_CENTRAL</code>	Solo servidor	API key del número central

Variables de reCAPTCHA (se deben agregar en Vercel aunque no están en `.env.example`):

Variable	Descripción
<code>NEXT_PUBLIC_RECAPTCHA_SITE_KEY</code>	Site key pública de Google reCAPTCHA v3
<code>RECAPTCHA_SECRET_KEY</code>	Secret key del servidor. Si no está, reCAPTCHA se omite en dev.

7. Estructura de carpetas

```

hacienda/
  docs/                • Documentación del proyecto
  public/
    logo-principal-fondo-claro.svg
    trebol-original.svg • Favicon
    placeholder-avatar.svg
    placeholder-evento.svg

```

```
src/
  app/
    page.tsx          •Home público (Server Component)
    sitemap.ts        •Sitemap dinámico App Router
    robots.ts        •robots.txt dinámico
    bodas/
    quince-anos/
    eventos-empresariales/
    revelacion-de-genero/
    update-password/  •Standalone (fuera del grupo auth)
    auth/confirm/     •Route Handler auxiliar
    (auth)/
      login/
      reset-password/
    portal/           •Portal clientes + staff
      dashboard/
      orden-servicio/
      actividades/
      mensajes/
      invitados/
      playlist/
      perfil/
      planner/
        nuevo-cliente/
        clientes/
          [clientId]/
            contrato/
            actividades/
            invitados/
            documentos/
            pagos/
            playlist/
            orden-servicio/[bookingId]/
            salon-mapas/
        asesor-comercial/
        asesor-logistica/
        gerente/
        staff/
    admin/
      dashboard/
      usuarios/
      clientes/
    editor/
      galeria/
      videos/
      imagenes-sitio/
      testimonios/
      paquetes/
      contenido/
  app/actions/        •Server Actions (sin API Routes)
    auth.ts
    contact.ts
    contactos-asesor.ts
    crear-cliente.ts
    contrato-aprobacion.ts
    contrato-items.ts
    orden-servicio.ts
    actividades.ts
    invitados.ts
    salon-mapas.ts
    pagos.ts
    documentos.ts
    playlist.ts
    admin/
      usuarios.ts
      generar-contrato.ts
    editor/
      galeria.ts
      videos.ts
      imagenes-sitio.ts
      testimonios.ts
      paquetes.ts
      contenido.ts
  components/
    home/
    events/           •NavBar, HeroSection, SliderGaleria, etc.
    portal/           •EventPageTemplate, Vista360, etc.
    portal/           •PortalShell, PortalSidebar, PortalHeader
```

contrato/	• ContratoPDF, ContractItemsForm, ContratoPlanner
admin/	• UsuariosManager, EventosManager, CambiarPasswordButton
asesor/	• ContactosAsesorView
clientes/	• ClientesTable (compartido admin/planner)
editor/	• GaleriaManager, VideosManager, etc.
contact/	• ContactForm, HomeContactForm
ui/	• SliderGaleria, WhatsAppIcon, CopyButton, etc.
lib/	
supabase/	
server.ts	• createClient() async SSR
client.ts	• createBrowserClient()
admin.ts	• createAdminClient() service_role
uploads/	
config.ts	• UPLOAD_KINDS, SITE_IMAGE_KEYS (sin "use server")
server.ts	• requestSignedUploadUrl, confirmUpload
client.ts	• uploadFileToSignedUrl (browser)
clientes.ts	
eventos.ts	
event-window.ts	• UPCOMING_EVENT_WINDOW_DAYS = 15
playlist-templates.ts	
random-slider.ts	
guest-count.ts	
salon-map-capacities.ts	
callmebot.ts	
contract-items.ts	
types/	
database.ts	• Tipos generados por Supabase CLI
proxy.ts	• Middleware Next.js 16 (exportado como "proxy")
supabase/	
migrations/	• 50+ migraciones en orden cronológico
next.config.ts	• HTTP security headers + imagen remota
tsconfig.json	
tailwind.config.ts	• Tailwind v4 (@theme inline)
.env.example	• Plantilla de variables de entorno

8. Flujos principales

8.1 Flujo de contacto público

1. Visitante llena formulario de contacto (Home o página de evento)
'Validación Zod (nombre, email, whatsapp /^[0-9]{9}\$/', mensaje)
'reCAPTCHA v3 (score mínimo)
2. SA contact.ts 'submitContactForm
'Inserta en contact_messages
3. Round-robin de asesores:
'Consulta asesor_assignments ORDER BY total_assignments ASC, last_assigned_at ASC NULLS FIRST
'Filtrar perfiles activos con rol asesor_comercial o wedding_planner
'Asigna el asesor con menos contactos
'Incrementa total_assignments del asesor seleccionado
4. Notificación CallMeBot (fire & forget):
'Si CALLMEBOT_PHONE_CENTRAL + API_KEY_CENTRAL configurados: envía WhatsApp al número central
'Nunca bloquea la respuesta al usuario
5. Asesor ve el contacto en /portal/asesor-comercial
'Puede cambiar estado: unread 'read 'en_proceso 'replied
'Botón wa.me con mensaje prellenado: nombre + tipo evento + fecha + teléfono

8.2 Flujo de onboarding de cliente (planner)

1. Planner va a /portal/planner/nuevo-cliente
'Llena datos: nombre, email, contraseña, tipo de evento, fecha, horario, espacio, invitados

```
2. SA crear-cliente.ts 'createClientAction
' Crea usuario en Supabase Auth (admin API)
' Crea fila en profiles
' Valida solapamiento de horario (start1 < end2 AND start2 < end1, medianoche normalizada)
' Crea booking con status 'confirmed'
' NOTA: la orden de servicio NO se inicializa aquí

3. Planner es redirigido a /portal/planner/clientes/[clientId]/contrato

4. Cliente puede acceder al portal con sus credenciales
```

8.3 Flujo de contrato

```
1. Planner llena prerequisites en /portal/planner/clientes/[clientId]/contrato:
' Verifica: CC, dirección, teléfono, email, valor_total, valor_anticipo del cliente
' Configura ítems del contrato (ContractItemsForm): DJ, catering, decoración, etc.

2. Planner genera PDF (botón "Generar contrato"):
' SA generarContratoPDF
' Valida prerequisites consolidados
' Genera PDF server-side con @react-pdf/renderer (renderToBuffer)
' Upload directo a Supabase Storage bucket 'documents' (server-side, sin signed URL)
' Nombre: "{TipoEvento} {DD-MM-YYYY} {NombreCliente}.pdf"
' Inserta registro en tabla documents

3. Cliente ve el contrato en /portal/documentos
' Puede "Aprobar" o "Solicitar ajustes"

4. Si cliente aprueba 'SA aprobarContrato:
' Verifica ownership (cliente solo aprueba su propio booking)
' contract_locked = true (el planner ya no puede editar ítems)
' initialize_service_order(booking_id) ' crea orden de servicio desde plantilla
' Notifica planners y admins

5. Si cliente solicita ajustes 'SA solicitarAjustesContrato:
' Notifica al staff para revision
' contract_locked permanece false

6. Para eliminar historial (solo cuando contract_locked=false):
' SA eliminarHistorialContratos
' Borra archivos de Storage
' Borra registros en tabla documents de tipo 'contrato'
```

8.4 Flujo de orden de servicio

```
1. Se inicializa automáticamente cuando el cliente aprueba el contrato
' initialize_service_order(booking_id): función PL/pgSQL idempotente
' Crea secciones e ítems desde plantilla según event_type
' Pre-llena 6 campos de Cabecera desde el booking

2. Planner completa los ítems en /portal/planner/orden-servicio/[bookingId]
' SA savePlannerItems ' filled_by='planner'

3. Música (playlist) se lee en la orden en modo solo lectura
' ORDEN_MUSIC_FIELD_MAP en playlist-templates.ts

4. La sección "Aprobación" (event_type='all', sort 99) es filled_by='client'
' Cliente aprueba la orden completa
' SA approveServiceOrder

5. Vista cliente: /portal/orden-servicio
' Muestra barra de progreso
' Muestra todas las secciones excepto "Aprobación" ordenadas por sort_order
' Botón "Aprobar orden"
```

9. Storage buckets

Bucket	Visibilidad	Límite	MIME types	Ruta de archivos
gallery	Público	10 MB	JPEG, PNG, WebP, GIF, SVG	{category}/{file-name}
videos	Público	500 MB	MP4, WebM, Quick-Time, MPEG	{filename}
documents	Privado	50 MB	PDF, DOC, DOCX	{booking_id}/{file-name}
avatars	Público	2 MB	JPEG, PNG, WebP	{user_id}/{file-name}

Políticas de acceso

- **gallery / videos:** Lectura pública. Escritura solo admin.
- **documents:** Lectura para cliente (solo sus propios bookings) o staff/admin. Escritura para staff/admin. Eliminación solo admin.
- **avatars:** Lectura pública. Cada usuario solo puede escribir/eliminar su propio directorio ({user_id}/).

10. Instrucciones de deploy

10.1 Prerrequisitos

- Node.js e 20
- pnpm instalado globalmente (`npm i -g pnpm`)
- Acceso al proyecto en Vercel
- Acceso al proyecto en Supabase

10.2 Deploy en Vercel

```
# 1. Clonar el repositorio
git clone <url-repo>
cd hacienda

# 2. Instalar dependencias
pnpm install

# 3. Configurar variables de entorno locales
cp .env.example .env.local
# Editar .env.local con los valores reales

# 4. Correr en desarrollo
pnpm dev

# 5. Build de producción (para verificar antes de deploy)
pnpm build
```

Para producción en Vercel:

1. Ir al dashboard de Vercel ' proyecto ' Settings ' Environment Variables 2. Agregar todas las variables de la sección 6 con sus valores reales 3. El deploy se activa automáticamente al hacer push a `main` 4. Para un deploy manual: conectar el repositorio en Vercel y usar la rama `main`

Configuración de dominio:

- Dominio registrado: `hacienda-encanto.com`
- Dominio con `www`: `www.hacienda-encanto.com` (principal)
- Configurado en Vercel ' Domains

10.3 Migraciones de Supabase

Las migraciones son archivos SQL en `supabase/migrations/` con nombre `YYYYMMDDNNNNN_descripcion.sql`.

Aplicar migraciones en producción:

```
# Instalar Supabase CLI (si no está instalado)
npm i -g supabase

# Login
supabase login

# Vincular al proyecto
supabase link --project-ref oewqyckeqlrpbjevap

# Ver migraciones pendientes
supabase db diff

# Aplicar migraciones
supabase db push
```

Para crear una nueva migración:

```
# Crear archivo de migración (el timestamp se genera automáticamente)
supabase migration new nombre_de_la_migracion

# Editar el archivo generado en supabase/migrations/
# Luego aplicar:
supabase db push
```

Regenerar tipos TypeScript después de cada migración:

```
supabase gen types typescript --project-id oewqyckeqlrpbjevap > src/types/database.ts
```

11. Mantenimiento

11.1 Agregar un nuevo usuario del equipo

1. Ir a `/admin/usuarios` con una cuenta admin 2. Clic en "Crear usuario" 3. Completar: nombre, email, contraseña, rol (cualquiera excepto `client`) 4. El usuario recibe sus credenciales y puede iniciar sesión en el portal

Via SA (código): `crearUsuario()` en `src/app/actions/admin/usuarios.ts`. Crea usuario en Auth + perfil en BD.

11.2 Cambiar contraseña de un usuario

Via interfaz admin:

- `/admin/usuarios` ' buscar usuario ' botón "Cambiar contraseña"
- `/admin/clientes/[clientId]` ' misma opción para clientes

Via código: `cambiarPassword(userId, newPassword)` en `admin/usuarios.ts`.

11.3 Crear un cliente nuevo

Solo planner o admin pueden crear clientes: 1. `/portal/planner/nuevo-cliente` 2. Completar formulario con datos del evento 3. El sistema valida solapamiento de horarios automáticamente 4. Se envía al flujo de contrato directamente

11.4 Actualizar contenido del sitio

Desde `/editor/contenido`` (rol editor o admin):

- Datos de la hacienda (nombre, representante, NIT, dirección, cuenta bancaria, etc.)
- Cláusulas del contrato (textos de las 12 cláusulas)
- Tour 360° (URL de Kuula cuando esté disponible)
- Firma del representante legal

Desde `/editor/galeria``:

- Subir, archivar, reordenar y eliminar imágenes de galería (máx 8 por categoría en Publicadas)

Desde `/editor/videos``:

- Gestionar videos del hero del Home y páginas de eventos

Desde `/editor/imagenes-sitio``:

- 8 imágenes editables: cards de eventos en Home, sección Nosotros, servicios (catering, fotografía, decoración)

Desde `/editor/testimonios``:

- CRUD de testimonios, incluyendo foto del cliente

Desde `/editor/paquetes``:

- Editar nombre y descripción de paquetes de servicio (sin precios públicos)

11.5 Registrar asesores en CallMeBot

Para que un asesor reciba notificaciones WhatsApp personales: 1. El asesor envía desde su WhatsApp el mensaje `I allow callmebot.com to send me messages` al número `+1 (347) 798-2047` 2. Recibirá una API key por respuesta 3. El admin la ingresa en el campo `callmebot_api_key` del perfil del asesor en `/admin/usuarios`

Para el número central: 1. Mismo proceso con el número central de la hacienda 2. Las keys se configuran como `CALLMEBOT_PHONE_CENTRAL` y `CALLMEBOT_API_KEY_CENTRAL` en Vercel ' Environment Variables

11.6 Verificaciones periódicas de Supabase

Revisar en el dashboard de Supabase:

Punto	Configuración esperada
Authentication ' Rate Limits	Configurados adecuadamente
Authentication ' JWT expiry	3600 segundos (1 hora)

Storage ' bucketdocuments	public: false (privado)
Database ' RLS	Habilitado en todas las tablas, especialmente login_attempts
Authentication ' Email Templates ' Reset Password	Link usa {{ .ConfirmationURL }}
Authentication ' URL Configuration ' Redirect URLs	Incluir https://www.hacienda-encanto.com/update-password y http://localhost:3000/update-password

11.7 Agregar o editar espacios/salones

Actualmente solo via SQL o acceso directo a la tabla spaces en Supabase. No existe interfaz UI dedicada para esto. Modificar directamente en el dashboard de Supabase ' Table Editor spaces.

12. Seguridad

Headers HTTP (next.config.ts)

Aplicados a todas las rutas (source: "/*")

Header	Valor
X-Frame-Options	DENY — previene clickjacking
X-Content-Type-Options	nosniff — previene MIME sniffing
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	camera=(), microphone=(), geolocation=(), payment=()
Content-Security-Policy	Ver next.config.ts para el CSP completo

Content Security Policy

- default-src 'self'
- script-src 'self' 'unsafe-eval' 'unsafe-inline' google.com gstatic.com (requeridos por Next.js App Router y reCAPTCHA)
- img-src 'self' data: blob: oewqyckeqlrpjbjevap.supabase.co
- media-src 'self' blob: oewqyckeqlrpjbjevap.supabase.co
- object-src 'none'
- form-action 'self'

Principios de seguridad implementados

- **RLS en toda la BD:** Ninguna tabla es accesible sin las políticas correctas
- `createAdminClient()` con service_role solo en Server Actions, nunca en cliente
- **No precios públicos:** El campo packages.price fue eliminado (migración 20260722) — era legible con anon key via REST
- **Constantes compartidas** nunca en archivos "use server" — viven en lib/ independientes

-
- **Archivos nunca pasan por Vercel** — signed URL pattern para todos los uploads
 - **Rate limiting en login:** tabla `login_attempts` + delay progresivo
 - **Validación Zod** en todos los Server Actions en el borde del sistema
-

Documento generado el 2026-07-29. Para dudas técnicas contactar al Ing. Jeisson Rincón.
